Gestion RH du laboratoire

Description de l'architecture

Octobre 2026

Résumé

Ce document décrit l'architecture de l'application de gestion des ressources humaines du laboratoire : pile technique, découpage en couches, modèle de données, conventions des contrôleurs, recherche générique, et invariants à respecter lors des évolutions. Il complète le `README.md` et le `CLAUDE.md` du dépôt.

Table des matières

1 Objectif et périmètre **2**

2 Pile technique **2**

3 Vue d'ensemble en couches **2**

4 Organisation du code **3**

5 Modèle de données **3**
5.1 Entités . . . 3
5.2 Relations . . . 3
5.3 Tables de jointure . . . 4
5.4 Décisions de modélisation . . . 4

6 Couche web **5**
6.1 Pattern CRUD commun . . . 5
6.2 Résolution explicite des relations . . . 5
6.3 Listes de chaînes . . . 5
6.4 Vues . . . 5

7 Recherche globale **6**

8 Persistance et configuration **6**

9 Invariants et pièges connus **6**
9.1 Pas de cascade sur les collections non resoumises . . . 6
9.2 Les côtés inverses ne s'éditent pas . . . 7
9.3 Toute nouvelle relation est bidirectionnelle . . . 7

10 Limites et évolutions possibles **7**

1 Objectif et périmètre

L'application gère les personnes d'un laboratoire de recherche et tout ce qui s'y rattache : structures (équipes, services), contrats, employeurs et grades, postes occupés, locaux (campus, bâtiments, bureaux), adresses et pays, publications.

Les choix de la version 1 sont volontairement minimalistes :

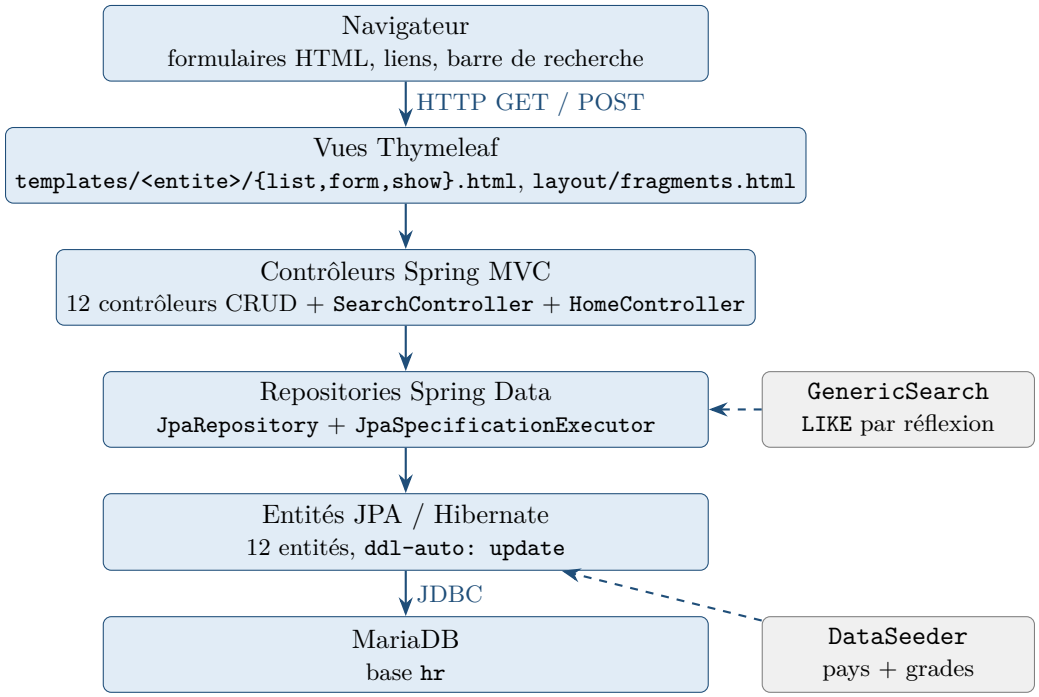
— une application web monolithique rendue côté serveur, sans frontend séparé ni API REST ;
— pas d'authentification ;
— toute relation du modèle est navigable dans les deux sens depuis l'interface, comme l'exige le cahier des charges ;
— tout le vocabulaire métier (classes, champs, routes, templates) est en français, pour coller au cahier des charges.

2 Pile technique

Composant	**Rôle**
Java 21	Langage et environnement d'exécution
Spring Boot 4.1	Socle applicatif (starters `webmvc`, `data-jpa`, `thymeleaf`, `validation`, `devtools`)
Spring Data JPA / Hibernate	Mapping objet-relationnel, repositories, `Specification` pour la recherche
Thymeleaf	Rendu HTML côté serveur
Jakarta Bean Validation	Contraintes déclaratives (`@NotBlank`)
MariaDB 11	Base de données, installée sur le poste (Homebrew)
Maven	Build (wrapper `mvnw` fourni)

3 Vue d'ensemble en couches

L'application suit une architecture MVC classique en trois couches, sans couche « service » intermédiaire : la logique métier est réduite et les contrôleurs parlent directement aux repositories.



La flèche entre le navigateur et les vues résume l'aller-retour HTTP : le contrôleur reçoit la requête, prépare le `Model` et renvoie le nom d'un template que Thymeleaf rend en HTML. `open-in-view` est activé, ce qui permet aux templates de parcourir les collections paresseuses (pages de détail listant les entités liées) pendant le rendu.

4 Organisation du code

```
src/main/java/fr/lab/hr/
  HrApplication.java   point d'entrée Spring Boot
  model/               12 entités JPA + enum Genre + interface Identifiable
  repository/          12 interfaces Spring Data JPA
  controller/          12 contrôleurs CRUD + SearchController + HomeController
  search/              GenericSearch, SearchHit, SearchResultGroup
  config/              DataSeeder (tables de référence)
src/main/resources/
  application.yml      connexion MariaDB, options JPA et Thymeleaf
  templates/           list/form/show par entité, layout/, search/
```

Toutes les entités implémentent `Identifiable` (une seule méthode `getId()`), ce qui permet à la recherche de construire des liens vers n'importe quel résultat sans connaître son type.

5 Modèle de données

5.1 Entités

Entité	Contenu
`Personne`	Identité (nom, prénom, prénom d'usage, nom de jeune fille, genre), badge, naissance, doctorat, statuts (HDR, associé, collaborateur), quotité, ORCID, idHAL, section CNRS, listes de téléphones, e-mails et pages web
`Structure`	Nom, structure parente, sous-structures, bureaux
`Contrat`	Nom, porteur, sous-porteurs, coordinateurs, partenaires, autres personnes
`Employeur`	Nom, public ou non, grades proposés
`Grade`	Code et libellé (IE, IR, AI, DR, CR, MCF, PR pré-remplis)
`Poste`	Personne, employeur, grade, dates de début et de fin
`Bureau`	Nom, capacité, bâtiment
`Batiment`	Nom, campus
`Campus`	Nom
`Adresse`	Rue, ville, code postal, latitude/longitude, pays
`Pays`	Code ISO et nom (249 pays pré-remplis)
`Publication`	Titre, année, identifiant HAL

5.2 Relations

La figure 1 montre les relations. Chaque flèche part du *côté propriétaire* (celui qui porte `@JoinColumn` ou `@JoinTable`, et donc dont le formulaire édite la relation) et pointe vers le côté inverse (`mappedBy`, lecture seule).

Toutes les relations sont mappées dans les deux sens : chaque page de détail affiche et lie les entités associées, quelle que soit la direction.

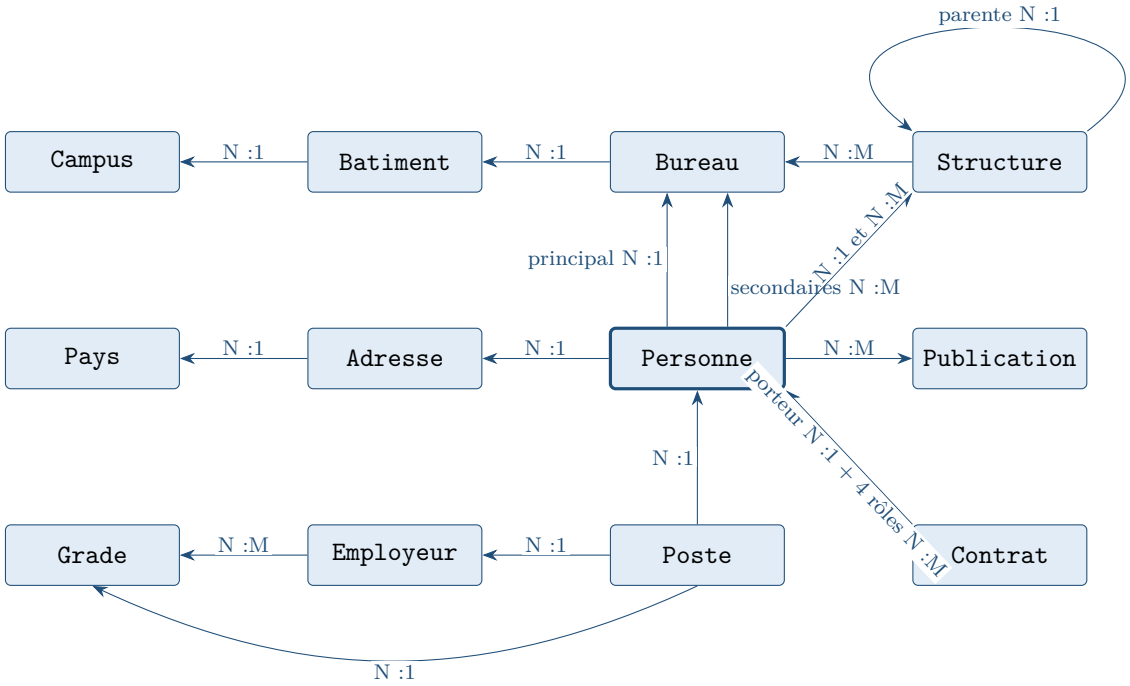


FIGURE 1 – Modèle de données. Les flèches partent du côté propriétaire de chaque relation.

5.3 Tables de jointure

Table	Propriétaire	Vue inverse
`personne_bureau_secondaire`	`Personne.bureauxSecondaires`	`Bureau.personnesSecondaire`
`personne_structure_impliquee`	`Personne.structuresImpliquees`	`Structure.personnesImpliquees`
`personne_publication`	`Personne.publications`	`Publication.auteurs`
`structure_bureau`	`Structure.bureaux`	`Bureau.structures`
`employeur_grade`	`Employeur.grades`	`Grade.employeurs`
`contrat_sous_porteur`	`Contrat.sousPorteurs`	`Personne.contratsSousPortes`
`contrat_coordinateur`	`Contrat.coordinateurs`	`Personne.contratsCoordines`
`contrat_partenaire`	`Contrat.partenaires`	`Personne.contratsPartenaire`
`contrat_autre_personne`	`Contrat.autresPersonnes`	`Personne.contratsAutre`

Les listes de chaînes de `Personne` sont des `@ElementCollection` stockées dans `personne_telephone`, `personne_email` et `personne_page_web`.

5.4 Décisions de modélisation

Bureaux d'une personne. Un bureau principal (`ManyToOne`) et une liste de bureaux secondaires (`ManyToMany`), le cahier des charges mentionnant les deux sans les distinguer.

Identité. Ajout de `nom`/`prenom` en plus de `nomJeuneFille`/`prenomUsage`. L'affichage utilise le prénom d'usage s'il est renseigné.

Employeurs et grades. Plutôt que de sous-classer EPST et Université en dur, chaque `Employeur` choisit ses `Grade` dans un catalogue commun. La règle « les employeurs publics définissent des statuts » reste ainsi modifiable depuis l'interface.

Contrats. Cinq relations distinctes vers `Personne`, une par rôle, toutes possédées par `Contrat`.

Publications. Entité minimale (titre, année, idHAL). Les auteurs se gèrent depuis la fiche de la personne, côté propriétaire.

6 Couche web

6.1 Pattern CRUD commun

Les 12 contrôleurs suivent exactement le même schéma, illustré ici pour `/personnes` :

Méthode	Route	Action
GET	`/personnes`	liste (`personne/list`)
GET	`/personnes/new`	formulaire de création (`personne/form`)
GET	`/personnes/{id}/edit`	formulaire de modification (`personne/form`)
GET	`/personnes/{id}`	page de détail (`personne/show`)
POST	`/personnes`	création ou modification selon le champ caché `id`
POST	`/personnes/{id}/delete`	suppression

Après un `save` ou un `delete`, le contrôleur redirige vers la liste (pattern Post/Redirect/Get). En cas d'erreur de validation, le formulaire est réaffiché avec les listes de référence nécessaires aux `<select>`.

6.2 Résolution explicite des relations

Il n'y a pas de `Converter<String, Entity>` global. Chaque formulaire poste les identifiants des entités liées dans des paramètres séparés, que le contrôleur résout lui-même :

```
@PostMapping
public String save(@Valid @ModelAttribute("personne") Personne personne,
        BindingResult result,
        @RequestParam(required = false) Long bureauPrincipalId,
        @RequestParam(required = false) List<Long> bureauxSecondairesIds,
        ...) {
    personne.setBureauPrincipal(bureauPrincipalId != null
            ? bureauRepository.findById(bureauPrincipalId).orElse(null) : null);
    personne.setBureauxSecondaires(bureauxSecondairesIds != null
            ? bureauRepository.findAllById(bureauxSecondairesIds)
            : Collections.emptyList());
    ...
    repository.save(personne);
    return "redirect:/personnes";
}
```

Ce choix rend le comportement de chaque formulaire lisible sans connaître la configuration globale de Spring, au prix d'un peu de répétition.

6.3 Listes de chaînes

Les listes simples (téléphones, e-mails, pages web) sont éditées dans un `<textarea>`, une valeur par ligne, et découpées côté serveur par `linesToList`. Aucun JavaScript n'est nécessaire.

6.4 Vues

Chaque entité possède trois templates : `list.html` (tableau, liens d'ajout et de suppression), `form.html` (création et modification, `<select>` simples ou multiples pour les relations) et `show.html` (détail, avec les entités liées cliquables dans les deux sens). La barre de navigation et la barre de recherche sont factorisées dans `layout/fragments.html`.

7 Recherche globale

Aucune requête de recherche n'est écrite à la main. `GenericSearch` construit par réflexion une `Specification` JPA qui applique un `LIKE` insensible à la casse sur tous les champs `String` déclarés de l'entité :

```
public static <T> Specification<T> containing(Class<T> entityClass, String term) {
    String pattern = "%" + term.toLowerCase() + "%";
    return (root, query, cb) -> {
        List<Predicate> predicates = new ArrayList<>();
        for (Field field : entityClass.getDeclaredFields()) {
            if (field.getType().equals(String.class)) {
                predicates.add(cb.like(cb.lower(root.get(field.getName())), pattern));
            }
        }
        if (predicates.isEmpty()) {
            return cb.disjunction();
        }
        return cb.or(predicates.toArray(new Predicate[0]));
    };
}
```

`SearchController` (`GET /recherche?q=...`) applique cette spécification aux 12 repositories, regroupe les résultats par type (`SearchResultGroup` de `SearchHit`) et masque les groupes vides. Une nouvelle entité devient cherchable en l'ajoutant simplement à la liste de `SearchController.rechercher()`.

La recherche ne porte que sur les champs texte directs : chercher le nom d'une structure fera remonter la structure, pas les personnes qui y sont rattachées.

8 Persistance et configuration

- **Schéma.** `spring.jpa.hibernate.ddl-auto: update` : Hibernate crée et fait évoluer le schéma au démarrage. Il n'y a pas de migrations Flyway/Liquibase.
- **Base locale.** MariaDB installée sur le poste (Homebrew), port 3306, base `hr`, utilisateur `hr`/`hr`, créés par `scripts/creer_base_dev.sql`.
- **Données de référence.** `DataSeeder` (`ApplicationRunner`) remplit `pays` à partir de `Locale.getISOCountries()` (noms en français) et `grade` avec IE, IR, AI, DR, CR, MCF et PR. Il ne fait rien si les tables contiennent déjà des données.
- **Développement.** Cache Thymeleaf désactivé et `spring-boot-devtools` pour le rechargement à chaud.

```
brew services start mariadb   # MariaDB locale
mvn spring-boot:run      # application sur http://localhost:8080
```

9 Invariants et pièges connus

9.1 Pas de cascade sur les collections non resoumises

Chaque `POST` reconstruit l'entité à partir du formulaire via `@ModelAttribute`. Les collections absentes du formulaire reviennent donc à leur valeur par défaut, en général une liste vide. Si une telle collection porte `cascade = CascadeType.ALL, orphanRemoval = true`, Hibernate supprime silencieusement toutes les lignes enfants à la sauvegarde suivante du parent.

C'est arrivé avec `Personne.postes` : chaque enregistrement d'une personne effaçait ses postes. La correction a été de retirer cascade et orphanRemoval ; `postes` est désormais une vue inverse en lecture seule, les postes étant gérés par leur propre contrôleur.

Règle : si l'entité enfant a son propre contrôleur CRUD, ne pas mettre de cascade côté parent.

9.2 Les côtés inverses ne s'éditent pas

Les collections `mappedBy` (par exemple `Bureau.personnesPrincipal` ou `Publication.auteurs`) sont uniquement affichées. Une relation se modifie toujours depuis le formulaire de l'entité propriétaire.

9.3 Toute nouvelle relation est bidirectionnelle

Le cahier des charges exige la navigation dans les deux sens : toute relation ajoutée doit être mappée des deux côtés, éditée via des paramètres `xxxId`/`xxxIds` côté propriétaire, et affichée sur les deux pages de détail.

10 Limites et évolutions possibles

— Pas d'authentification ni de gestion des droits (voulu pour la v1).
— `ddl-auto: update` ne convient pas à la production : passer à des migrations versionnées (Flyway) avant tout déploiement réel.
— Le formulaire `Poste` propose tous les grades, sans les filtrer selon ceux de l'employeur choisi.
— La recherche ne traverse pas les relations.
— Les tests automatisés se limitent au test de chargement du contexte (`HrApplicationTests`) ; toute règle métier non triviale ajoutée devrait être accompagnée d'un test.